

Microservices Distributed Transactions

Introduzione: Fondamenti e ACID

Le tre domande

1. Come eseguire operazioni che si estendono su più servizi?
2. Come ottenere la coerenza dei dati tra questi servizi?
3. Come gestire guasti e rollback in un ambiente distribuito?

★ "Il nostro problema principale è che **non possiamo usare le tradizionali transazioni ACID di un singolo database tra microservizi distribuiti**. Dobbiamo trovare pattern e principi diversi."

La roadmap dei pattern

Pattern	Cosa risolve
Saga (Choreography / Orchestration)	Coordinare transazioni multi-servizio
Transactional Outbox	Pubblicare eventi in modo affidabile dopo un commit locale
Compensating Transaction	Annullare gli effetti dei passaggi precedenti
Dual Write Problem	Il principio da capire per giustificare gli altri
CDC (Change Data Capture)	Reagire alle modifiche efficientemente — Debezium + Kafka

Ripasso: cos'è una transazione

Una sequenza di una o più operazioni di database (insert, update, delete) eseguite come una singola unità logica di lavoro.

★ "La caratteristica principale è il **principio "tutto o niente"**. Se tutte le operazioni hanno successo, **la transazione viene committata** e le modifiche diventano permanenti. Se una qualsiasi operazione fallisce, **l'intera transazione viene annullata (rollback)** e il database torna allo stato precedente. **Nessuna delle modifiche viene applicata.**"

Le proprietà ACID

Proprietà	Significato	Esempio
A Atomicity	Tutto o niente	"Una transazione si completa completamente o non si completa affatto"
C Consistency	Il database rimane valido	"Passa da uno stato valido a un altro"
I Isolation	Le transazioni concorrenti non interferiscono	"Due transazioni in esecuzione affiancate senza conflitti"
D Durability	Le modifiche persistono	"I dati restano salvati anche in caso di riavvio del server"

*"Ma questi funzionano in un'architettura monolitica con un unico database. Realizzare transazioni ACID tra moduli logici diversi è relativamente semplice, **perché tutto avviene all'interno di un unico database**. Ma cosa succede se dobbiamo gestire transazioni tra diversi microservizi e diversi database?"*

The Challenge of Distributed Transactions in Microservices

Il confronto

Nel MONOLITE (Lezione 3):

```
BEGIN TRANSACTION
  | elabora il pagamento
  | decrementa le scorte
  | crea il record dell'ordine
COMMIT      ← tutto insieme, o rollback di tutto
```

*"Tutti questi aggiornamenti avvengono **nel contesto di un singolo database**, e il database relazionale fornisce **forti garanzie ACID**."*

Nei MICROSERVIZI:

```
[Ordering] → Orders DB      (SQL Server)
[Payment]  → Payments DB    (PostgreSQL)
[Inventory] → Inventory DB   (MongoDB)
           ↑
           nessun COMMIT comune esiste
```

*"Una singola transazione di business richiede ora **cambiamenti coerenti tra questi database multipli, indipendenti e potenzialmente di tipi diversi**."*

⚠ Perché il Two-Phase Commit (2PC) non è la risposta

*"I meccanismi transazionali per le transazioni distribuite, come il **Two-Phase Commit (2PC)**, che cerca di estendere le proprietà ACID su più database, **non sono generalmente adatti né consigliati per i microservizi**."*

*"Introducono problemi significativi: **complessità, problemi di prestazioni, disponibilità ridotta e accoppiamento stretto**."*

Perché in pratica: il 2PC ha un **coordinatore** che chiede a tutti i partecipanti "sei pronto?" (fase 1) e poi "commit!" (fase 2). **Durante l'attesa, tutti i partecipanti tengono i lock aperti**. Se il coordinatore cade tra le due fasi, i partecipanti restano

bloccati indefinitamente. È l'opposto dell'autonomia e della disponibilità che cerchiamo nei microservizi.

★ Due tipi di problema, due soluzioni diverse

La lezione fa una distinzione fondamentale.

CASO A — Propagazione di un cambiamento (risolvibile con eventi)

Scenario: il prezzo di un prodotto cambia, e Basket ha una copia nel carrello.

"Il microservizio Catalog non può scrivere direttamente nel database di Basket — violerebbe il Database-per-Service. E anche una chiamata sincrona a ogni cambio di prezzo non è scalabile e reintroduce accoppiamento."

Soluzione: sincronizzazione event-driven (Lezione 9 e 11).

"Catalog pubblica `ProductPriceChangedIntegrationEvent`, Basket si iscrive e aggiorna i prezzi locali. Si ottiene eventual consistency, e questo va bene per molti scenari."

CASO B — Transazione di scrittura multi-step atomica (NON resolvibile con eventi soli) ★

Scenario: piazzare un ordine.

"Dobbiamo creare un ordine, poi elaborare il pagamento, poi aggiornare le scorte, poi spedizione, notifica... Abbiamo bisogno che tutti questi passaggi riescano efficacemente come un'unica unità logica."

"Se il pagamento fallisce, non dovremmo avere un ordine marcato come pagato né un decremento delle scorte. Se le scorte non possono essere decrementate dopo il pagamento, dobbiamo rimborsare il pagamento e annullare l'ordine."

★ *"Questi requisiti "tutto o niente" su più servizi indipendenti sono il cuore del problema delle transazioni distribuite. La semplice eventual consistency, pur essendo utile per la propagazione dei dati, non è in grado di gestire da sola l'atomicità di un flusso di lavoro di business così complesso."*

Il punto da cogliere: propagare un cambiamento e coordinare una transazione sono cose diverse. Gli eventi bastano per il primo; per il secondo serve una Saga.

Bounded Context for Transactional Boundaries

Il collegamento con la Lezione 6

*"Un **bounded context** traccia un confine logico chiaro intorno a una parte specifica del dominio. All'interno di questo confine, i modelli, il linguaggio e le regole sono espliciti e coerenti. Spesso progettiamo i microservizi allineandoli a questi bounded context."*

"All'interno di ciascun microservizio, la coerenza dei dati locali può essere gestita in modo relativamente semplice usando le tradizionali transazioni ACID contro il proprio database dedicato."

"Tuttavia, la sfida sorge quando un caso d'uso end-to-end richiede azioni e cambiamenti di stato tra PIÙ di questi microservizi e i loro bounded context."

★ Definizione: Transactional Boundary (confine transazionale)

L'ambito di un'operazione di business che deve raggiungere un risultato complessivo coerente, anche se coinvolge più servizi indipendenti, ciascuno con le proprie transazioni ACID locali.

"Questi confini si estendono a servizi che, per loro stessa concezione, sono a basso accoppiamento e hanno i propri data store. Non possiamo applicare le tradizionali proprietà ACID a questi servizi distribuiti usando meccanismi come il 2PC, a causa di prestazioni, disponibilità e accoppiamento."

"Quando un'operazione attraversa questi confini, in genere si punta all'eventual consistency del processo di business complessivo. Usiamo pattern per coordinare una sequenza di transazioni locali all'interno di ogni servizio partecipante."

Perché identificarli è cruciale:

"Ci indica l'insieme MINIMO di servizi che devono coordinarsi con successo per completare uno specifico caso d'uso end-to-end. L'idea è trovare la più piccola unità di atomicità di business che abbraccia i servizi, e progettare comunicazione e logica di compensazione intorno a quel confine."

Il caso concreto: Order Fulfillment

CONFINE TRANSAZIONALE

1. [Ordering]	crea record ordine	← ACID locale	
2. [Payment]	elabora pagamento	← ACID locale	
3. [Inventory]	decrementa scorte	← ACID locale	
4. [Shipping]	organizza consegna	← ACID locale	
5. [Notification]	invia conferma	← ACID locale	

tutte gestite come UNA transazione di business

"Ogni singolo microservizio gestisce perfettamente le proprie transazioni locali, ma COORDINARLE per ottenere un risultato complessivo coerente è la sfida."

I due percorsi:

- **Success path:** tutte le transazioni locali committano → ordine completato

- **Failure path:** *"se la transazione locale di Inventory fallisce, devono attivarsi transazioni di compensazione: rimborsare il pagamento e annullare l'ordine"*
-

SAGA Pattern (Choreography and Orchestration)

Definizione

Una Saga è una **SEQUENZA DI TRANSAZIONI LOCALI**. L'operazione di business complessiva viene suddivisa in una serie di fasi, dove ogni fase è una transazione locale eseguita da un singolo microservizio contro il proprio database.

Il meccanismo:

1. Ogni transazione locale è **ACID-compliant** nel proprio servizio
2. *"Quando una transazione locale si completa con successo, la saga innesca la transazione locale successiva, spesso pubblicando un evento o inviando un comando"*
3. ★ *"Se una qualsiasi transazione della sequenza fallisce, la saga impone l'esecuzione di **TRANSAZIONI DI COMPENSAZIONE** — operazioni volte ad annullare il lavoro svolto dalle precedenti transazioni completate con successo"*

*"L'obiettivo è riportare il sistema a uno stato coerente dal punto di vista del **BUSINESS**."*

Nota la differenza sottile ma cruciale: una Saga **non garantisce isolamento** (la "I" di ACID). Durante la saga, uno stato intermedio è visibile agli altri: l'ordine esiste ma non è ancora pagato. Il monolite lo nascondeva; la Saga no. Questo va progettato nel modello di business (stati: Pending, Confirmed, Failed).

★ Stile 1 — Choreography-based Saga (coreografia)

Ogni microservizio comunica con gli altri scambiando eventi tramite message broker. Non c'è un coordinatore centrale.

"Con la coreografia, ogni microservizio esegue la propria transazione locale e pubblica un evento al broker, innescando così transazioni locali in altri microservizi."

Il flusso di successo:

```
[Ordering]  transazione locale → pubblica OrderPlaced
           ↓
[Payment]   ascolta OrderPlaced → addebita → pubblica PaymentProcessed
           ↓
[Inventory] ascolta PaymentProcessed → riserva scorte → pubblica StockReserved
           ↓
[Shipping]  ascolta StockReserved → ...
```

Il flusso di fallimento:

"Se Inventory incontra un errore durante la prenotazione, **pubblica un evento di FALLIMENTO**. In questo caso, i **microservizi eseguono transazioni di compensazione per annullare l'addebito e cancellare l'ordine**."

I 3 vantaggi:

Vantaggio	Dettaglio
Decentralizzazione e flessibilità	"Ogni microservizio reagisce agli eventi. Disaccoppia la dipendenza diretta dei microservizi nella gestione delle transazioni "
Nessun single point of failure ★	"Non c'è un orchestratore . Le responsabilità sono distribuite tra i partecipanti"
Semplicità per flussi semplici	"Va bene se non richiede troppe fasi"

I 2 svantaggi:

Svantaggio	Dettaglio
Complessità di gestione ★	"Il flusso di lavoro della saga diventa confuso quando si aggiungono nuove fasi. Può richiedere maggiore coordinamento"
Dipendenze cicliche tra eventi	"Esiste il rischio di dipendenza da consumo di eventi ciclici tra i partecipanti"

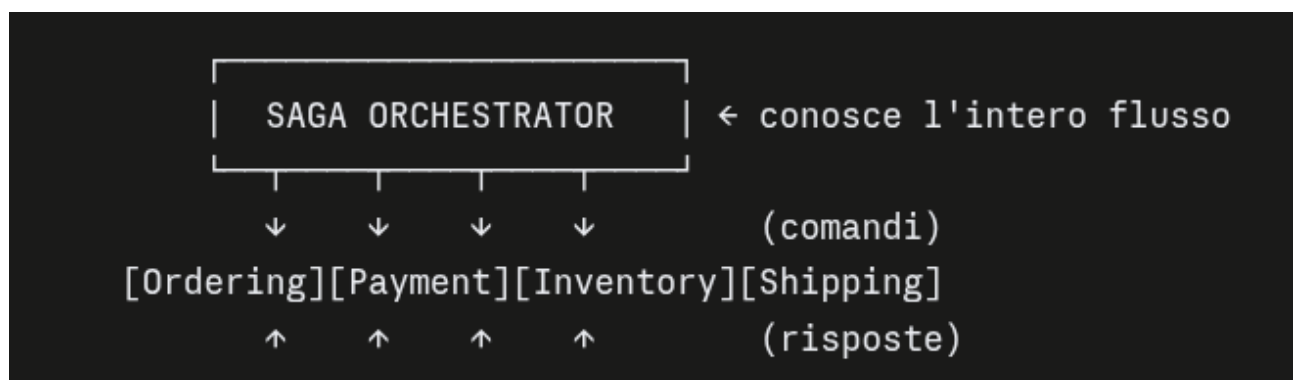
★ "La saga basata sulla **coreografia** è buona per flussi di lavoro semplici, se non richiedono troppe fasi."

Il problema reale della coreografia: con 6 servizi, **nessuno conosce il flusso completo**. Per capire cosa succede dopo PaymentProcessed devi cercare nel codice di tutti i servizi chi è sottoscritto a quell'evento. Il processo di business **non esiste in nessun punto del codice** — è emergente.

★ Stile 2 — Orchestration-based Saga (orchestrazione)

Un servizio **orchestratore centrale** coordina e gestisce le singole transazioni.

"L'orchestratore è responsabile di **avviare la transazione e garantire che ogni passo venga eseguito nell'ordine corretto**. Se uno dei passi fallisce, l'orchestratore può usare le transazioni di compensazione per fare rollback e ripristinare il sistema."



"Questo microservizio **controllore centralizzato orchestra il flusso di lavoro e invoca l'esecuzione delle transazioni locali in sequenza**. Se una fase fallisce, **esegue i passaggi di rollback con transazioni di compensazione**."

Il vantaggio principale:

"Fornisce un punto di controllo chiaro e centralizzato. Questo facilita la comprensione e il debug del sistema, nonché l'aggiunta di nuove transazioni o la modifica di quelle esistenti."

I 2 svantaggi:

"L'orchestratore può diventare un single point of failure: se si guasta, l'intero sistema potrebbe non essere in grado di completare le transazioni. Inoltre può diventare un collo di bottiglia se il sistema è molto carico, poiché tutte le transazioni devono passare attraverso di esso."

★ *"L'orchestrazione è adatta a flussi di lavoro complessi che comprendono molti passaggi."*

Il confronto

	Choreography	Orchestration
Coordinatore	Nessuno — eventi	Orchestratore centrale
Dove vive il processo	Emergente, distribuito nei servizi	Esplicito, in un posto solo
Accoppiamento	✓ Minimo	⚠ I servizi dipendono dall'orchestratore
SPOF	✓ Nessuno	⚠ L'orchestratore
Debug	✗ Difficile	✓ Più facile
Adatto a	Flussi semplici, pochi passi	Flussi complessi, molti passi

Compensating Transaction Pattern

Definizione

Un'operazione che ha lo scopo specifico di annullare l'effetto di una transazione precedentemente completata con successo all'interno di una saga.

*"Si può pensare che sia il **Ctrl+Z** di una particolare fase del processo di business distribuito."*

*"Il rollback si ottiene eseguendo una transazione di compensazione corrispondente per ogni fase completata, di solito **NELL'ORDINE INVERSO** della loro esecuzione."*

```
Forward:      Crea ordine → Paga → [Aggiorna scorte] × FALLISCE
Compensating:      ← Rimborso ← Annulla ordine
                  (ordine inverso)
```

Le 4 caratteristiche

1. Specifica per l'operazione

*"Per **ogni** transazione della saga che apporta un cambiamento di stato, **dovete progettare e implementare una transazione di compensazione corrispondente.**"*

2. Deve essere IDEMPOTENTE ★ — "molto importante"

*"La stessa transazione di compensazione potrebbe fallire — ad esempio per un problema di rete durante il tentativo di rimborso — e il sistema potrebbe riprovare. Quindi deve essere idempotente: **applicarla più volte deve dare lo stesso risultato che applicarla una sola volta.**"*

*"Il rimborso di un pagamento deve avvenire **UNA SOLA VOLTA**, anche se la compensazione viene attivata più volte a causa dei retry."*

3. Può fallire anch'essa

*"Anche una transazione di compensazione può incontrare errori. La logica della saga deve gestire questi insuccessi, **spesso con retry o segnalando il problema per un intervento MANUALE** se i retry falliscono ripetutamente."*

4. Raggiunge eventual consistency

*"I dati di tutti i microservizi coinvolti **potrebbero non essere perfettamente ripristinati immediatamente.**"*

★ Il punto concettuale più importante

*"È importante notare che le transazioni di compensazione **NON** ripristinano necessariamente l'esatto stato precedente del database, record per record, come fa il rollback di un database. Piuttosto, eseguono **OPERAZIONI DI BUSINESS CHE NEGANO SEMANTICAMENTE l'effetto della transazione originale.**"*

Esempio che rende chiara la differenza: un rollback di database cancella la riga del pagamento come se non fosse mai esistita. Una **compensazione** invece **emette un rimborso**: nel database restano **due righe** (addebito + rimborso). Il saldo è lo stesso, **la storia è diversa**. E dal punto di vista contabile è **più corretto così**.

Le 5 sfide

Sfida	Dettaglio
Complessità ★	<i>"Definire la logica esatta per annullare semanticamente un'operazione può essere complesso. Cosa significa annullare una spedizione già partita dal magazzino? Le regole di business per la compensazione devono essere molto chiare"</i>
Accesso ai dati originali	<i>"Per rimborsare un pagamento servono l'ID della transazione originale e l'importo. Questi dati devono essere conservati e accessibili alla logica di compensazione"</i>
Garantire l'idempotenza	<i>"Se una chiamata di rimborso viene ritentata più volte, non deve emettere rimborsi multipli"</i>
Fallimenti nelle compensazioni	<i>"Cosa succede se la compensazione stessa fallisce — es. il gateway di pagamento è down? Serve solida logica di retry e un meccanismo per segnalare il problema per intervento manuale"</i>
Ordine di esecuzione	<i>"Tipicamente nell'ordine inverso della transazione in avanti"</i>

Problem: The Dual Write Problem

Questo è uno dei problemi più sottili e importanti dell'intero corso.

Definizione

Il problema del dual write si presenta quando una singola operazione di business deve apportare modifiche di stato a DUE O PIÙ sistemi esterni distinti, e si vuole che queste modifiche avvengano atomicamente — entrambe riescono, o nessuna delle due.

L'esempio classico:

*"Persistere un cambiamento di stato **in un database**, e poi, come parte della stessa operazione logica, **pubblicare un evento a un message broker** per notificare altri microservizi."*

★ *"Il problema si verifica perché **NON esiste una transazione distribuita globale che possa abbracciare sia un database tradizionale sia un message broker per garantire l'atomicità di entrambe le scritture.**"*

! I due scenari, entrambi rotti

Scenario 1 — Prima il database, poi l'evento

1. ✓ COMMIT ordine su Orders DB
2. ✨ CRASH (o il broker è down)
3. ✗ L'evento OrderCreated NON viene pubblicato

Risultato: abbiamo un ordine nel database, ma nessun evento è stato inviato. I servizi a valle non sanno del nuovo ordine. Il sistema è in uno stato incoerente."

Conseguenza concreta: l'ordine esiste ma non viene mai pagato, né spedito, né notificato. È un ordine "fantasma".

Scenario 2 — Prima l'evento, poi il database

1. ✓ PUBLISH OrderCreated
2. ✨ Il commit del database FALLISCE (violazione vincoli, DB down)

"Risultato: l'evento è stato pubblicato e altri microservizi potrebbero iniziare a reagire a un ordine che in realtà **NON ESISTE.**"

Conseguenza concreta: il cliente viene addebitato, le scorte vengono decrementate, la spedizione parte — per un ordine che non è mai stato salvato.

L'esigenza

"Abbiamo bisogno di un meccanismo affidabile che garantisca che, se un cambiamento viene committato nel nostro database, alla fine venga pubblicato l'evento corrispondente. E viceversa: se la modifica al database fallisce, nessun evento deve essere pubblicato."

"Dobbiamo collegare queste due operazioni in modo molto più robusto rispetto alla semplice esecuzione sequenziale nel codice."

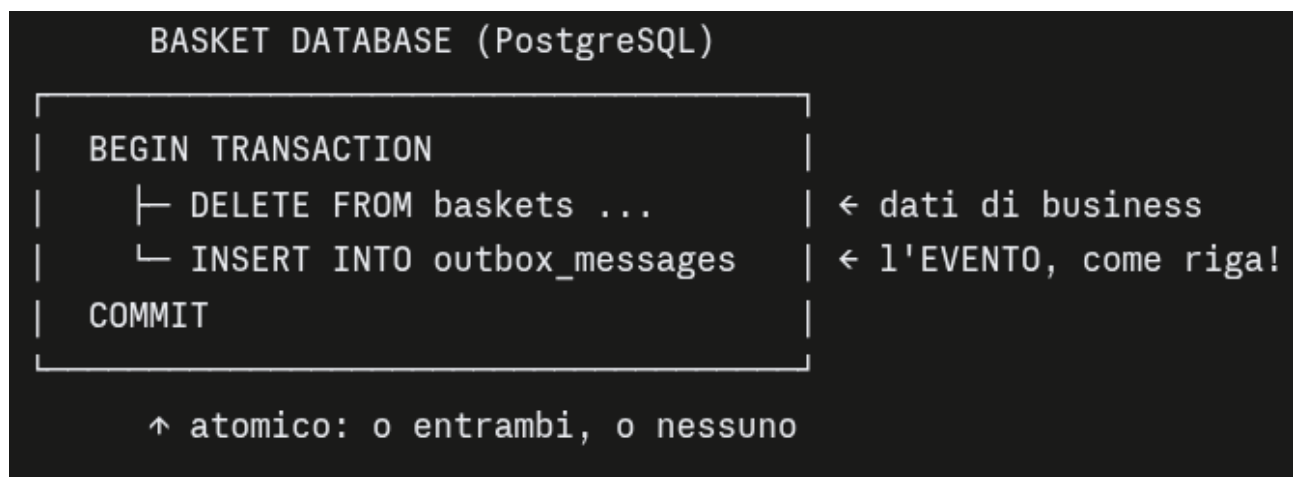
→ Transactional Outbox e CDC.

Best practice citata: *"usiamo Red Hat, Apache Kafka e CDC con Debezium nelle applicazioni event-driven. Oppure database come CockroachDB, che ha il CDC incorporato."*

Transactional Outbox Pattern

L'idea centrale

Invece di inviare immediatamente l'evento a un broker esterno, il microservizio scrive SIA l'aggiornamento dei dati di business SIA il record dell'evento ALL'INTERNO DELLA STESSA TRANSAZIONE DI DATABASE LOCALE.



"Questo garantisce che non ci sia MAI una modifica dei dati di business senza un corrispondente record di evento, e viceversa."

"La tabella outbox funge essenzialmente da coda temporanea e persistente per gli eventi in uscita, memorizzata insieme ai dati di business."

Le 2 fasi

FASE 1 — Commit atomico

"Sia le modifiche alle entità sia gli eventi vengono salvati all'interno di una singola transazione locale."

FASE 2 — Pubblicazione asincrona

*"Un processo o componente indipendente, spesso chiamato **Message Relay** o **Event Publisher**, è responsabile dell'invio effettivo degli eventi al broker."*

[Message Relay]

- | polla periodicamente la tabella outbox per nuovi record non elaborati
- | tenta di pubblicarli al broker
- └ al successo, MARCA l'evento come "processed" (o lo cancella)

*"Questa separazione garantisce che le transazioni di business critiche **NON** dipendano direttamente dalla disponibilità o dal successo della pubblicazione su un broker esterno in quel preciso momento."*

I 4 vantaggi

Vantaggio	Dettaglio
Risolve il dual write ★	<i>"Se i dati di business vengono salvati, l'evento si troverà nella outbox. Se la transazione fallisce, non viene salvata nessuna delle due."</i>
Affidabilità della pubblicazione	<i>"Il relay può implementare robusta logica di retry. Se il broker non è temporaneamente disponibile, gli eventi restano al sicuro nella tabella outbox finché non possono essere pubblicati"</i>
Prestazioni	<i>"La transazione originale comporta solo la scrittura sul database locale, che è molto veloce. L'operazione più lenta — pubblicare su un broker attraverso la rete — viene rinviata al processo in background"</i>
Coerenza dei dati	<i>"L'evento pubblicato si basa su dati effettivamente committati, assicurando che i servizi a valle ricevano informazioni accurate"</i> <i>"È particolarmente importante quando si lavora con dati critici — transazioni finanziarie, elaborazione di ordini e-commerce — dove non ci si può permettere di perdere eventi."</i>

Le 4 considerazioni

1. Implementazione del Message Relay

*"Può essere **un semplice background worker o thread** che polla periodicamente. Per scenari avanzati, si possono usare tecniche di CDC."*

2. At-least-once delivery e consumer idempotenti ★

*"Il relay garantisce tipicamente la consegna **at-least-once**: un evento sarà pubblicato almeno una volta, ma in rari scenari di fallimento potrebbe essere pubblicato di nuovo. È quindi fondamentale che i consumer a valle siano **IDEMPOTENTI**."*

3. Eventual consistency

*"Poiché gli eventi sono pubblicati da un processo separato **dopo** il commit, c'è un ritardo intrinseco."*

4. Manutenzione della tabella outbox ★

"La tabella outbox crescerà nel tempo. Serve una strategia per ripulire periodicamente gli eventi già processati, per evitare che diventi troppo grande."

Problem: Listen and Polling Outbox Table

Il pattern Outbox funziona, ma il **polling** ha i suoi difetti.

I 3 problemi del polling

1. Spreco di risorse

*"Il Message Relay **interroga continuamente la tabella outbox**. Per la maggior parte del tempo, soprattutto durante i periodi di calma, questa query non restituirà alcun nuovo evento. Queste query vuote continuano a consumare CPU, I/O e risorse di rete del database — il che è inefficiente."*

2. Latenza nella pubblicazione

*"Gli eventi **non vengono pubblicati istantaneamente**: devono attendere il prossimo ciclo di polling. Se l'intervallo è di 5 secondi, un evento potrebbe restare nella tabella per 5 secondi prima di essere recuperato."*

★ Il dilemma:

"Accorciare l'intervallo per ridurre la latenza aumenta lo spreco di risorse. È difficile trovare un intervallo di polling adatto a tutti: troppo corto → si sprecano risorse; troppo lungo → la propagazione degli eventi viene ritardata."

3. Un nuovo dual write, nel relay stesso ★

*"Il relay legge dal database e poi pubblica sul broker. Se il relay si blocca **DOPO aver letto ma PRIMA di aver pubblicato con successo e marcato come processato**, potrebbe perdere eventi o ripubblicarli."*

*"È **un punto sottile**, spesso mitigato marcando gli eventi come "in corso" prima della pubblicazione. Ma resta comunque una complessità e un rischio di dual write."*

Nota l'ironia: il pattern Outbox risolve il dual write dell'applicazione, ma **lo reintroduce nel relay**.

CDC: Change Data Capture with Outbox Pattern

Definizione

CDC è una tecnologia per catturare le modifiche incrementali (insert, update, delete) apportate ai dati in un database di origine, e fornirle ai sistemi a valle — invece di dover interrogare ripetutamente il database per rilevare le modifiche.

★ Come funziona: il transaction log

*"Molte soluzioni CDC funzionano **monitorando continuamente il TRANSACTION LOG** (noto anche come commit log o write-ahead log) del database di origine. Il **transaction log** è il luogo dove il database registra **TUTTE le modifiche PRIMA che vengano scritte nelle tabelle di dati effettive**."*

★ *"Attingendo a questo log, gli strumenti CDC possono estrarre gli eventi di modifica SENZA gravare significativamente sul database di origine e SENZA richiedere modifiche alle applicazioni che scrivono sul database."*

Questi sono i due punti chiave:

- **Overhead minimo** — il log il database lo mantiene comunque, per il recovery
- **Zero modifiche all'applicazione** — l'app non sa che esiste il CDC

★ **CDC + Outbox: la combinazione ottimale**

```
1. [Ordering] BEGIN TRANSACTION
    | INSERT INTO orders
    | INSERT INTO outbox_messages ← l'evento
  COMMIT
    ↓ (scritto nel transaction log)
2. [DEBEZIUM] monitora il transaction log
    | rileva la nuova riga in outbox_messages — QUASI Istantaneamente
    ↓
3. [DEBEZIUM] legge l'evento, lo trasforma, lo pubblica su Kafka
    ↓
4. [Inventory] [Notification] consumano dal topic Kafka
```

"Questo approccio elimina la necessità di pollare la tabella outbox. Il rilevamento degli eventi diventa quasi in tempo reale, e impone overhead minimo al database di origine."

★ *"È un modo molto più efficiente e scalabile di implementare il message relay. E il servizio applicativo non ha bisogno di conoscere Kafka o Debezium direttamente, dopo aver scritto nella sua tabella outbox."*

Definizione: Debezium — *"una piattaforma distribuita open source leader del settore, con connettori per MySQL, PostgreSQL, SQL Server, MongoDB, e integrazione perfetta con Kafka."*

CDC integrato nei database moderni

"La buona notizia è che la potenza del CDC viene sempre più integrata direttamente in molti database moderni, soprattutto cloud-native."

Database	Funzionalità
CockroachDB	"Database SQL distribuito con Change Feeds integrati, che possono trasmettere in streaming a Kafka"
Azure Cosmos DB	" Change Feed — un flusso continuo delle modifiche apportate agli elementi in un container"
Amazon DynamoDB	" DynamoDB Streams — catturano una sequenza ordinata nel tempo di modifiche a livello di elemento . Le applicazioni le leggono in tempo reale, es. usando AWS Lambda come trigger"

"Questo spesso elimina la necessità di strumenti CDC separati come Debezium, perché il database stesso fornisce il meccanismo."

Design: Microservices Architecture with SAGA, Outbox and CDC

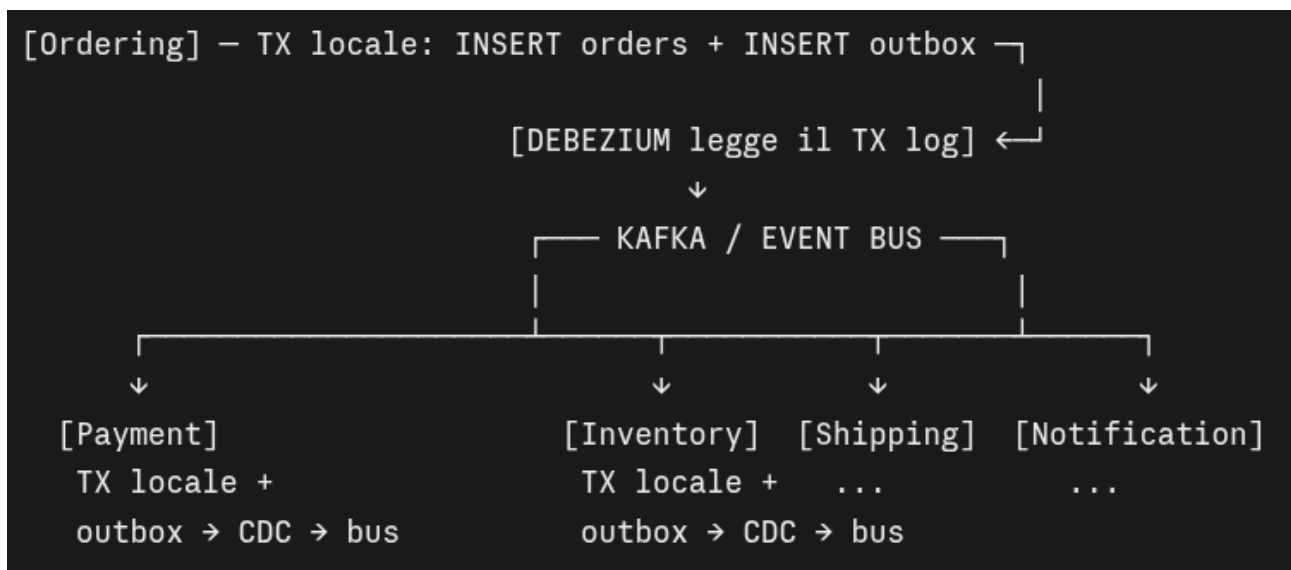
Toolkit aggiornato

Nuovi requisiti non funzionali:

1. **Coerenza dei dati tra i servizi** per l'intera transazione di business — **raggiunta alla fine** (eventual)
2. **Completamento affidabile o compensazione completa** delle transazioni multi-step
3. **Elevata tolleranza ai guasti** nei processi distribuiti
4. ★ **Evitare il problema del dual write**

Nuovi pattern: Saga (con scelta tra coreografia e orchestrazione), **Transactional Outbox**, **Compensating Transaction**, CDC.

★ L'architettura completa



Il flusso passo per passo:

Passo 1 — Avvio dell'ordine

*"Ordering riceve la richiesta (da Basket tramite BasketCheckoutEvent, o dal client tramite gateway). All'interno di una singola transazione ACID locale, crea il record dell'ordine E scrive l'evento **OrderCreated** nella propria tabella outbox. Queste operazioni devono avvenire nella stessa transazione, atomicamente."*

Passo 2 — Pubblicazione via CDC

"Debezium monitora il log del database di Ordering, rileva la nuova voce nella tabella outbox, legge l'evento e lo pubblica sull'event bus (topic Kafka)."

Passo 3-5 — La Saga per coreografia

*"Payment sottoscrive **OrderCreated**, tenta il pagamento, **esegue la propria transazione locale aggiornando la tabella payments** E scrivendo l'evento nella propria **outbox**. Il CDC del DB di Payment cattura e pubblica **PaymentProcessed**."*

*"Inventory sottoscrive **PaymentProcessed**, riserva le scorte, **stesso pattern: transazione locale + outbox + CDC → pubblica StockReserved**."*

*"La coreografia event-driven continua per **Shipping e Notification**."*

★ *Nota l'eleganza: ogni servizio ripete **lo stesso identico pattern: transazione locale (business + outbox) → CDC → evento**. È una regola uniforme, applicabile ovunque.*

La gestione dei fallimenti:

*"Se Inventory fallisce, **pubblica StockReservationFailed**. Payment consuma questo evento di fallimento e avvia la transazione di compensazione **RefundPayment**. Ordering marca l'ordine come fallito."*

Scelte tecnologiche

Componente	Opzioni
Event bus	Apache Kafka — "estremamente popolare per l'event streaming, adatto alle saghe grazie ai suoi log ordinati e duraturi ". RabbitMQ — "robusto, soprattutto per messaggistica tradizionale e routing". Cloud: AWS EventBridge/SNS, Azure Service Bus/Event Grid
CDC	Debezium (open source, leader). Oppure CDC integrato : CockroachDB Change Feeds, Cosmos DB Change Feed, DynamoDB Streams

I 3 vantaggi

1. **Affidabilità delle transazioni distribuite** — "le operazioni multi-step si completano con successo su tutti i servizi, **o sono adeguatamente compensate**"
2. **Pubblicazione affidabile degli eventi** ★ — "Outbox + CDC **risolvono efficacemente il problema del dual write**"
3. **Servizi altamente disaccoppiati** — "comunicano **solo attraverso eventi via message broker**"

★ I 4 svantaggi

1. Aumento sostanziale della complessità

*"**Molte più parti mobili**: la logica della saga, la tabella outbox in ogni servizio, i connettori CDC, il broker, i consumer di eventi. **Un aumento sostanziale della complessità architetturale e operativa**."*

2. Debug e tracing estremamente difficili

*"**Seguire una singola transazione di business mentre scorre attraverso più eventi e servizi può essere molto difficile**. **Distributed tracing robusto, correlation ID e logging completo sono ASSOLUTAMENTE ESSENZIALI**."*

3. Complessità delle transazioni di compensazione

*"Progettare compensazioni **corrette, affidabili e idempotenti** per ogni fase è un compito non banale."*

4. Costi operativi

*"Distribuire, gestire e monitorare strumenti CDC, cluster di broker e il flusso di eventi complessivo **richiede competenze operative e strumenti significativi.**"*

Code: Microservices with SAGA, Outbox Pattern and CDC

Cartella 11-eshop-microservices-outbox. Il pattern è implementato nel **microservizio Basket**, lato produttore, per il **checkout**.

Nota: l'implementazione usa un **Outbox Processor con polling** (un background service), **non** CDC/Debezium. La lezione insegna il CDC concettualmente, ma il codice mostra la versione più semplice.

L'AppHost

```
var basketDb = postgres.AddDatabase("basketdb"); // ← ora Basket ha un DB PostgreSQL
```

```
builder.AddProject<Basket>("basket")
    .WithReference(basketDb) // tabelle business + tabella OUTBOX
    .WithReference(cache) // Redis
    .WithReference(messaging); // RabbitMQ
```

*"Basket ora ha il suo **basketdb PostgreSQL**, dove si trovano le tabelle di business e ora anche la tabella outbox."*

★ Lato produttore: BasketService.CheckoutBasket

```
var strategy = dbContext.Database.CreateExecutionStrategy();
await strategy.ExecuteAsync(async () =>
{
    using var transaction = await dbContext.Database.BeginTransactionAsync(); // ← APRE

    // logica di business
    var basket = await GetBasket(userName);
    basketCheckout.TotalPrice = basket.TotalPrice;

    // 1) crea l'evento e lo SCRIVE NELLA OUTBOX (non lo pubblica!)
    var integrationEvent = new BasketCheckoutIntegrationEvent { ... };
```

```

var outboxMessage = new OutboxMessage
{
    Type    = integrationEvent.GetType().AssemblyQualifiedName,
    Content = JsonSerializer.Serialize(integrationEvent), // serializzato
    OccurredOn = DateTime.UtcNow,
    ProcessedOn = null // ← non ancora elaborato
};
dbContext.OutboxMessages.Add(outboxMessage);

// 2) cancella il carrello (business)
dbContext.ShoppingCarts.Remove(basket);
await cache.RemoveAsync(userName);

await dbContext.SaveChangesAsync();
await transaction.CommitAsync(); // ← COMMIT ATOMICO di 1) e 2)
});

```

★ *"Invece di pubblicare direttamente l'evento su RabbitMQ, creiamo un'entità OutboxMessage, serializziamo l'evento di integrazione nella colonna **Content**, e salviamo questo messaggio nella stessa transazione in cui rimuoviamo il carrello."*

"In questo modo salviamo atomicamente entrambe le operazioni. Se una parte fallisce, l'intera transazione viene annullata e non viene salvato alcun messaggio outbox. Così si risolve il problema del dual write."

★ Il consumatore della outbox: OutboxProcessor

Percorso: Basket/Data/Processors/OutboxProcessor.cs — un **BackgroundService**.

```

protected override async Task ExecuteAsync(CancellationToken stoppingToken)
{
    while (!stoppingToken.IsCancellationRequested)
    {
        // 1. cerca i messaggi NON elaborati
        var messages = await dbContext.OutboxMessages
            .Where(m => m.ProcessedOn == null) // ← la condizione chiave
            .ToListAsync();
    }
}

```

```

foreach (var message in messages)
{
    // 2. deserializza nel tipo di evento originale
    var eventType = Type.GetType(message.Type);
    var eventMessage = JsonSerializer.Deserialize(message.Content, eventType);

    // 3. pubblica su RabbitMQ tramite MassTransit
    await bus.Publish(eventMessage, stoppingToken);

    // 4. marca come elaborato — per non ripubblicarlo
    message.ProcessedOn = DateTime.UtcNow;
}

await dbContext.SaveChangesAsync();
await Task.Delay(TimeSpan.FromSeconds(10), stoppingToken); // ← intervallo di polling
}
}

```

*"Un **background service** significa che si apre un processo separato dentro il **microservizio Basket** che esegue periodicamente questo metodo. Qui usiamo un ritardo di **10 secondi** — regolabile."*

*"Dopo aver pubblicato con successo, **aggiorniamo la colonna ProcessedOn** per evitare di rielaborare l'evento."*

Nota che qui si vede esattamente il "problema del polling" della Sezione 8: 10 secondi di latenza, e query ripetute a vuoto quando non c'è traffico.

Lato consumatore: Ordering/EventHandlers/BasketCheckoutEventHandler

*"Consuma l'evento **BasketCheckout** sottoscritto da RabbitMQ ed esegue l'operazione **creando un nuovo ordine e avviando il processo di order fulfillment**."*

★ **La demo end-to-end**

1. Nella webapp si aggiungono al carrello **bastoncini da trekking** e una **giacca antipioggia**
2. Si va al checkout, si compila il form, si clicca il pulsante
3. **Breakpoint 1** — nella Razor page → F5
4. **Breakpoint 2** — nel **YARP API Gateway** → F5
5. **Breakpoint 3** — **BasketService.CheckoutBasket**: si vede **aprire la transazione**, creare l'evento di integrazione, **serializzarlo in un record outbox**, e **rimuovere il carrello nella stessa transazione** → **commit**

6. **Dopo circa 10 secondi** ★ — **Breakpoint 4** nell'`OutboxProcessor`: trova un messaggio con `ProcessedOn = null`, lo itera, ottiene il tipo, deserializza, **pubblica su RabbitMQ**, imposta `ProcessedOn`
7. **Breakpoint 5** — il microservizio **Ordering** consuma l'evento e **crea l'ordine**

Le verifiche

RabbitMQ management UI: si vede l'evento `BasketCheckout` pubblicato dall'`outbox processor` e consumato da `Ordering`.

pgAdmin — la tabella `outbox_messages` ★:

Id	Type	Content (JSON)	OccurredOn
ProcessedOn			
----- ----- ----- -----			

1	BasketCheckoutIntegrationEvent...	{"userName":"sw",	14:22:01
14:22:11		"totalPrice":249.98, ...}	

← valorizzata!

Il contenuto è **in formato JSON** e include username, prezzo totale e tutti i dettagli dell'evento, **serializzati e deserializzati usando questa colonna.**"

*"ProcessedOn è stato valorizzato dalla nostra applicazione quando il processor ha letto il record, pubblicato l'evento, e poi impostato **ProcessedOn** per evitare di rielaborare l'evento."*

La traccia **Aspire**:

```
POST /basket-service/basket/checkout → [Gateway] → [Basket]
  ↳ DELETE dalla cache Redis
  ↳ INSERT INTO outbox_messages ← nella STESSA transazione

... 10 secondi dopo, traccia SEPARATA ...

[Outbox Processor] → SELECT da PostgreSQL → PUBLISH su RabbitMQ
                                     ↓
                                     [Ordering] CONSUME → INSERT in Orders DB
```

"Si può notare che eseguiamo l'**INSERT** nella tabella `outbox` nella stessa transazione della rimozione del carrello. In questo modo eseguiamo lo schema `outbox`, e un processo separato leggerà questi dati ed eseguirà il nostro evento di integrazione separatamente."

Problem: Database Operations are Expensive, Low Performance

Dove siamo

*"Abbiamo implementato **Outbox e CDC** per una pubblicazione affidabile degli eventi, il **pattern Saga** per le transazioni distribuite, e **CQRS e viste materializzate** per read model ottimizzati."*

★ *"Questi pattern sono eccellenti per coerenza, affidabilità e disaccoppiamento. Tuttavia, **NON** risolvono intrinsecamente tutti i problemi di prestazioni legati all'accesso ai dati."*

Il problema: "last mile latency"

"Il costo dell'accesso frequente al data store. La lentezza nell'accesso ai dati riduce le prestazioni dei microservizi."

"Anche con microservizi ben progettati e percorsi di comunicazione ottimizzati, le operazioni di database possono causare problemi."

I 4 sintomi

Problema	Dettaglio
Le operazioni di DB sono costose	<i>"Leggere e scrivere su database basati su disco comporta operazioni di I/O, ordini di grandezza più lente dell'accesso alla memoria. Anche query complesse su DB ben indicizzati richiedono tempo"</i>
Accesso ripetitivo agli stessi dati ★	<i>"Al microservizio Catalog potrebbero essere richiesti i dettagli dei 10 prodotti più popolari centinaia di volte al secondo. Recuperarli da PostgreSQL ogni singola volta è inefficiente"</i>
Latenza nell'elaborazione degli eventi	<i>"L'elaborazione di un evento da parte di un consumer può ancora comportare letture e scritture di database, che aumentano il tempo complessivo"</i>
Latenza nelle chiamate API sincrone	<i>"Se il servizio chiamato deve accedere al database per ogni richiesta, la latenza complessiva si accumula"</i>
★ La lacuna identificata	

*"Abbiamo appreso che l'output caching è utile per le pagine prodotto rivolte al cliente (Lezione 3), ma che dire dei dati ad accesso frequente **ALL'INTERNO o TRA i microservizi interni?**"*

*"Se un microservizio ha frequentemente bisogno di un dato, e recuperarlo dal suo storage primario richiede **anche solo poche decine o centinaia di millisecondi**, questo può limitare significativamente throughput e reattività."*

L'obiettivo

*"Abbiamo bisogno di **memorizzare e recuperare i dati ad accesso frequente in modo molto più rapido** rispetto ad andare ripetutamente al database principale."*

1. **Ridurre la latenza** delle operazioni di lettura più comuni
2. **Diminuire il carico sui data store primari**
3. **Migliorare reattività e throughput complessivi**

*"Come soluzione, impareremo **la cache distribuita**."*